

华中科技大学

课程实验报告

课程名称： 深度学习

实验名称： 实验二

院 系： 计算机科学与技术学院

专业班级： 本硕博 2301 班

学 号： U202315763

姓 名： 王家乐

2025 年 12 月 23 日

1. 任务要求

本实验要求围绕 CIFAR-10 数据集完成一个卷积神经网络（CNN）在图像分类任务中的设计、训练与分析。具体而言，需要自行设计一个至少包含卷积层和全连接层的 CNN 模型，对 CIFAR-10 中 $32 \times 32 \times 3$ 的彩色图像进行 10 类分类，并完成模型的训练与测试（训练集 50000 张，测试集 10000 张）。在实验过程中，尝试和比较不同的网络结构与关键超参数设置，如卷积核大小、通道数、池化方式、激活函数（如 ReLU、LeakyReLU）、是否引入 Batch Normalization 或 Dropout 等，并分析其对模型性能的影响。实验报告需给出完整的网络结构说明、训练过程（如按 epoch 的损失变化）、最终在训练集和测试集上的分类准确率，并结合实验结果对模型性能进行分析与总结。

2. 实验设计

2.1 实验环境

项	参数
操作系统	Windows11
CPU	12th Gen Intel(R) Core(TM) i7-12700H
RAM	16GB
编程语言	Python3.10.18
平台	VScode & Jupyter Notebook

2.2 相关依赖

依赖项	版本
torch	2.8.0
pandas	2.3.1
numpy	2.2.6
matplotlib	3.10.5
tqdm	4.67.1

2.3 数据集加载和处理

本实验采用 CIFAR-10 图像数据集作为训练与测试数据。通过自定义的 `load_cifar10_numpy` 函数，分别加载 CIFAR-10 的训练集和测试集，其中训练集包含 50000 张样本，测试集包含 10000 张样本。每张样本大小为 32×32 的三通道彩色图像，对应的类别标签为 0 至 9，共 10 个类别。数据加载完成后，图像数据与标签分别转换并存储为 `train_data`、`train_label`、`test_data` 和 `test_label`，其中图像数据以 `numpy.array` 形式保存，标签以 `int64` 类型的 `numpy.array` 保存，便于后续进行数值计算、模型训练与性能评估。

```
def load_cifar10_numpy(root="data"):
    """ 加载 CIFAR-10 数据集并转换为 Numpy 格式 """
    # 下载并加载训练集和测试集
    train_ds = CIFAR10(root=root, train=True, download=True)
    test_ds = CIFAR10(root=root, train=False, download=True)
    # 提取数据和标签并转换为 numpy 数组
    train_data = np.array(train_ds.data)          # (50000, 32, 32, 3)
    train_label = np.array(train_ds.targets).astype(np.int64)
    test_data = np.array(test_ds.data)            # (10000, 32, 32, 3)
    test_label = np.array(test_ds.targets).astype(np.int64)
    return train_data, train_label, test_data, test_label
# 加载数据
train_data, train_label, test_data, test_label = load_cifar10_numpy(root="data")
```

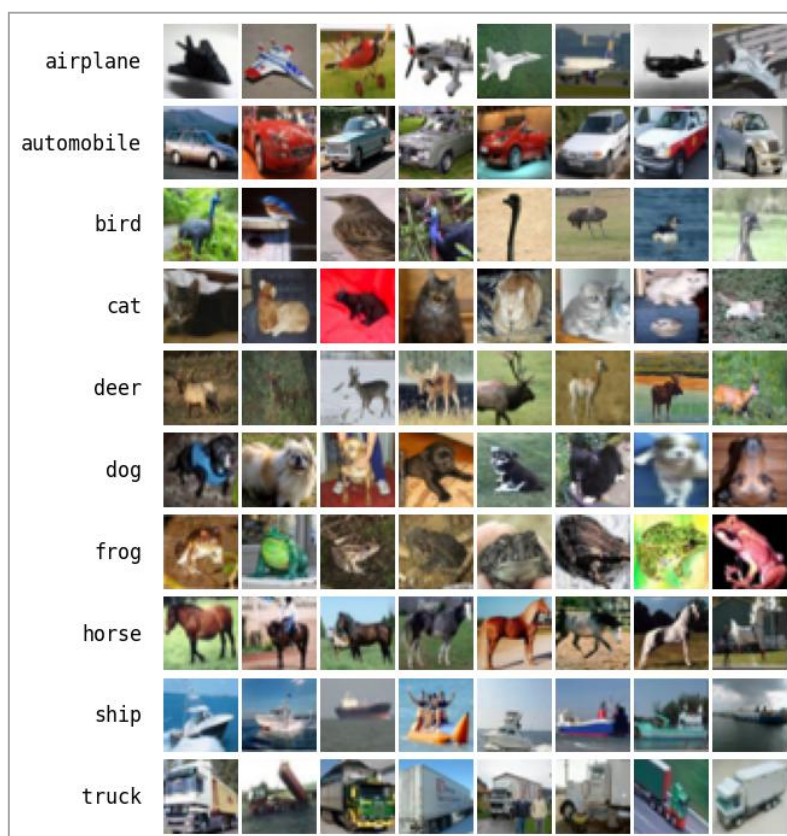
本实验基于 PyTorch 的 Dataset 接口自定义实现了 `CIFAR10Dataset` 类，用于统一管理 CIFAR-10 数据集的读取与预处理过程。该类在初始化阶段接收原始图像数据及其对应标签，并将标签统一转换为 `int64` 类型，以满足分类任务及损失函数的输入要求。在数据获取阶段，将原始的 $32 \times 32 \times 3$ 的 `uint8` 彩色图像转换为 `float32` 类型，并对像素值进行归一化处理，将其缩放至 $[0, 1]$ 区间，以提升模型训练的稳定性与收敛速度。针对训练集，在数据加载过程中引入随机水平翻转的数据增强策略，以增强模型的泛化能力。随后，将图像数据由 (H, W, C) 的格式转换为 (C, H, W)，以符合卷积神经网络对输入张量格式的要求。最终，

处理后的图像数据与标签均被转换为 PyTorch 的 Tensor 类型，便于通过 DataLoader 进行批量加载与后续模型训练与测试。

```
class CIFAR10Dataset(Dataset):
    """自定义Dataset类，用于PyTorch数据加载"""
    def __init__(self, data, labels, train=True):
        self.data = data
        self.labels = labels.astype(np.int64)
        self.train = train
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        # CIFAR-10: (H, W, C) uint8 -> (C, H, W) float32
        x = self.data[idx].astype(np.float32) / 255.0
        if self.train and np.random.rand() > 0.5:
            x = np.flip(x, axis=1).copy() # 沿宽度方向翻转
        x = np.transpose(x, (2, 0, 1))
        y = self.labels[idx]
        return torch.from_numpy(x), torch.tensor(y, dtype=torch.long)

# 实例化Dataset对象
train_dataset = CIFAR10Dataset(train_data, train_label, train=True)
test_dataset = CIFAR10Dataset(test_data, test_label, train=False)
```

从数据集中随机挑选部分图像进行可视化如下图所示：



2.4 卷积神经网络设计

卷积神经网络（CNN）由若干卷积模块与全连接层串联构成，通过卷积操作提取局部空间特征，并在池化层作用下降低特征图尺寸，从而实现对图像的层次化表征学习。本实验针对 CIFAR-10 的 $32 \times 32 \times 3$ 彩色图像，网络卷积部分采用可配置的多层卷积结构（默认通道数为 [32, 64]），每个卷积层使用大小为 3 的卷积核并采用 same padding；卷积后可选择加入 Batch Normalization，再经过指定的非线性激活函数（默认 ReLU）与池化层（默认 2×2 最大池化）进行特征变换与下采样，同时可按设定比例引入 Dropout2d（默认 0.2）以抑制过拟合。经过两次池化后，特征图空间尺寸由 32×32 变为 8×8 ，因此展平后的特征向量维度为 $64 \times 8 \times 8$ 。全连接部分同样采用可配置结构（默认仅包含一个 128 维隐藏层），各层之间激活函数默认为 ReLU，最终通过输出层映射到 10 个类别，实现 CIFAR-10 的分类预测。

```
class CNN(nn.Module):
    def __init__(
        self,
        conv_channels=[32, 64],
        kernel_size=3,
        pool_type='max',
        activation='relu',
        use_bn=True,
        dropout_rate=0.2,
        fc_sizes=[128],
        num_classes=10,
    ):
        super(CNN, self).__init__()
        # 保存配置
        self.conv_channels = conv_channels
        self.kernel_size = kernel_size
        self.pool_type = pool_type
        self.activation = activation
        self.use_bn = use_bn
        self.dropout_rate = dropout_rate
```

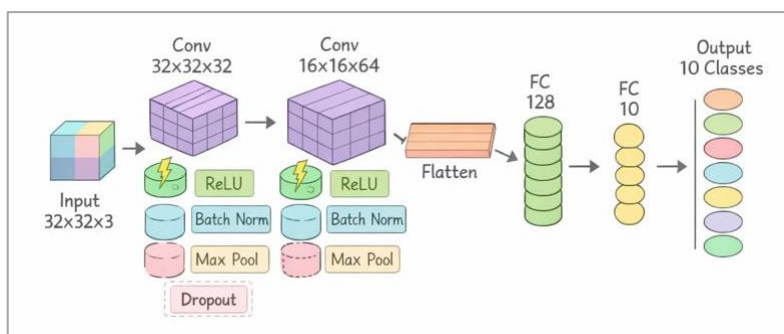
```
self.fc_sizes = fc_sizes
# 选择激活函数
def get_activation():
    if activation == 'relu':
        return nn.ReLU()
    elif activation == 'sigmoid':
        return nn.Sigmoid()
    elif activation == 'tanh':
        return nn.Tanh()
    elif activation == 'leaky_relu':
        return nn.LeakyReLU(0.1)
    elif activation == 'elu':
        return nn.ELU()
    else:
        return nn.ReLU()
# 选择池化层
def get_pool():
    if pool_type == 'max':
        return nn.MaxPool2d(kernel_size=2, stride=2)
    elif pool_type == 'avg':
        return nn.AvgPool2d(kernel_size=2, stride=2)
    else:
        return nn.MaxPool2d(kernel_size=2, stride=2)
# 构建卷积层
conv_layers = []
in_channels = 3 # CIFAR-10 是RGB 图像
padding = kernel_size // 2 # same padding
for out_channels in conv_channels:
    conv_layers.append(nn.Conv2d(in_channels, out_channels,
                                  kernel_size=kernel_size, padding=padding))
    if use_bn:
        conv_layers.append(nn.BatchNorm2d(out_channels))
    conv_layers.append(get_activation())
    conv_layers.append(get_pool())
    if dropout_rate > 0:
        conv_layers.append(nn.Dropout2d(dropout_rate))
    in_channels = out_channels
self.conv_layers = nn.Sequential(*conv_layers)
# 计算卷积层输出的特征图大小
# 输入: 32x32, 每次池化后尺寸减半
feature_size = 32 // (2 ** len(conv_channels))
self.flatten_size = conv_channels[-1] * feature_size * feature_size
# 构建全连接层
fc_layers = []
prev_size = self.flatten_size
```

```

for fc_size in fc_sizes:
    fc_layers.append(nn.Linear(prev_size, fc_size))
    if use_bn:
        fc_layers.append(nn.BatchNorm1d(fc_size))
    fc_layers.append(get_activation())
    if dropout_rate > 0:
        fc_layers.append(nn.Dropout(dropout_rate))
    prev_size = fc_size
fc_layers.append(nn.Linear(prev_size, num_classes))
self.fc_layers = nn.Sequential(*fc_layers)
# 前向传播
def forward(self, x):
    x = self.conv_layers(x)
    x = x.view(x.size(0), -1) # Flatten
    x = self.fc_layers(x)
    return x

```

本实验默认的网络结构如下图所示：



2.5 网络训练

首先根据配置信息创建模型。

```

def create_model(cfg):
    """ 根据配置字典创建模型实例 """
    model = CNN(
        conv_channels=cfg.get("conv_channels", [32, 64]),
        kernel_size=cfg.get("kernel_size", 3),
        pool_type=cfg.get("pool_type", "max"),
        activation=cfg.get("activation", "relu"),
        use_bn=cfg.get("use_bn", True),
        dropout_rate=cfg.get("dropout_rate", 0.2),
        fc_sizes=cfg.get("fc_sizes", [128]),
        num_classes=cfg.get("num_classes", 10),
    )
    return model.to(device)

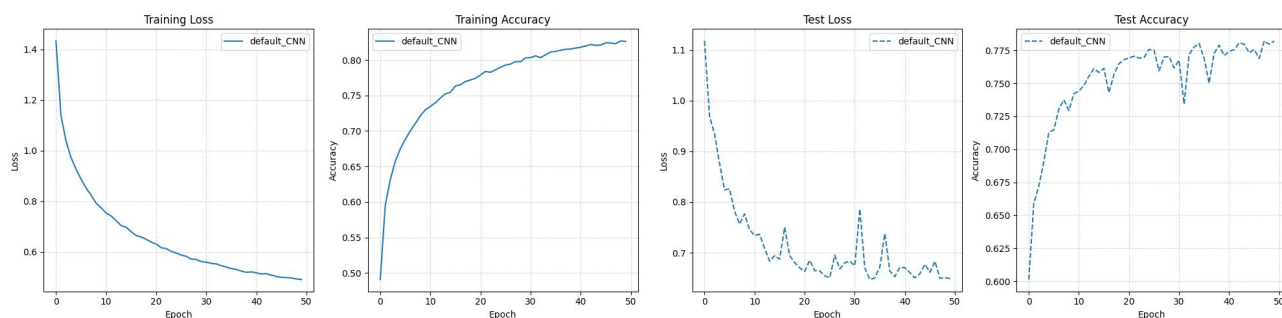
```

接下来训练模型，最大迭代次数（`num_epochs`）默认为 50，模型名称（`name`）、批大小（`batch_size`）、和学习率（`learning_rate`）从配置信息 `config` 中获取。使用 `DataLoader` 创建数据对象，本实验选用交叉熵（Cross Entropy）作为损失函数，选用 Adam（Adaptive Moment Estimation）作为优化器。训练过程中记录每个 epoch 在训练集和测试集上的损失和准确率，并将其写入 `output` 文件夹下的同名 `log` 文件中。此外，为了更为直观地反映网络训练全过程的性能变化和分类效果，在训练结束后通过 `matplotlib` 库对训练集和测试集的损失、准确率进行可视化展示。具体代码过长，这里不再展示。

3. 训练结果与分析

3.1 默认网络结构

如 2.4 节所述，默认网络结构为一个两层卷积、单隐藏层全连接的 CNN。卷积部分的通道数依次为 32 和 64，卷积核大小为 3，并使用最大池化在每个卷积块后进行 2×2 下采样；非线性变换采用 ReLU 激活函数。为提升训练稳定性与收敛速度，卷积层及全连接层均启用批归一化，同时设置 Dropout 比率为 0.2 以缓解过拟合。全连接部分默认包含一个 128 维隐藏层，最终输出 10 维分类 logits。训练相关的默认超参数为 batch size 128、learning_rate $1e-3$ 、num_epochs 50。训练结束后，得到训练过程的模型性能变化曲线如下图：



从实验结果可以看出，在默认网络结构与统一训练条件下，模型在训练集与测试集上的损失和准确率均表现出良好的收敛特性。随着训练轮数的增加，训练损失持续下降并逐渐趋于平稳，训练准确率稳步提升，最终达到约 82.62%，表明模型能够较充分地拟合训练数据；与此同时，测试损失整体呈下降趋势，测试准确率在一定波动后趋于稳定，最终达到约 78.21%。训练集与测试集性能之间保持合理差距，未出现明显的过拟合现象，说明所设计的默认 CNN 结构具备较好的泛化能力和训练稳定性，能够在 CIFAR-10 图像分类任务上取得较为理想的分类效果。default_CNN.log 文件如下：

```
Epoch,Train Loss,Train Accuracy,Test Loss,Test Accuracy
1,1.4341,0.4907,1.1177,0.6014
2,1.1397,0.5950,0.9693,0.6590
3,1.0393,0.6312,0.9342,0.6720
4,0.9732,0.6572,0.8765,0.6907
5,0.9276,0.6746,0.8239,0.7126
6,0.8879,0.6883,0.8261,0.7148
7,0.8525,0.7000,0.7819,0.7304
8,0.8248,0.7105,0.7565,0.7375
9,0.7929,0.7213,0.7766,0.7294
10,0.7744,0.7298,0.7453,0.7421
11,0.7537,0.7344,0.7340,0.7442
12,0.7423,0.7397,0.7363,0.7483
13,0.7236,0.7464,0.7086,0.7553
14,0.7040,0.7524,0.6832,0.7611
15,0.6976,0.7544,0.6941,0.7582
16,0.6799,0.7632,0.6873,0.7613
17,0.6647,0.7651,0.7502,0.7429
18,0.6590,0.7696,0.6944,0.7567
19,0.6497,0.7718,0.6801,0.7648
20,0.6386,0.7742,0.6702,0.7678
21,0.6312,0.7789,0.6630,0.7692
22,0.6163,0.7838,0.6851,0.7705
23,0.6132,0.7827,0.6646,0.7691
24,0.6020,0.7860,0.6641,0.7695
25,0.5953,0.7897,0.6545,0.7756
26,0.5877,0.7929,0.6506,0.7752
27,0.5824,0.7942,0.6950,0.7594
28,0.5712,0.7978,0.6681,0.7699
```

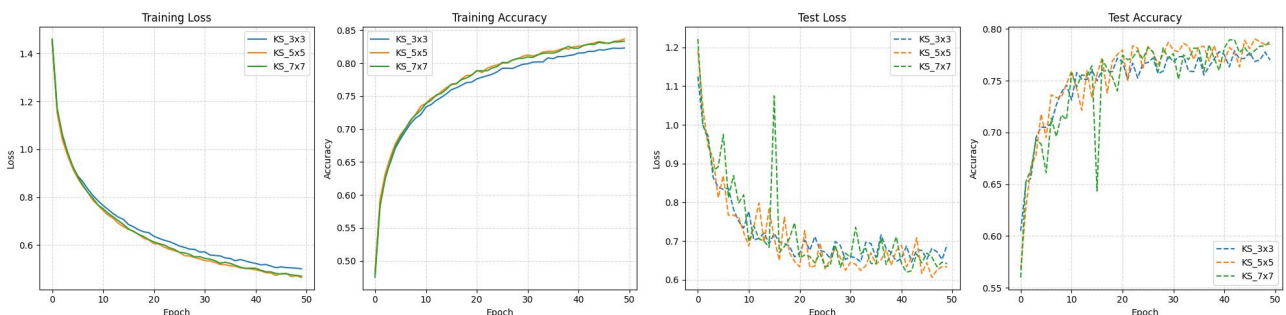
```

29,0.5701,0.7974,0.6801,0.7702
30,0.5615,0.8030,0.6832,0.7617
31,0.5593,0.8033,0.6744,0.7675
32,0.5546,0.8059,0.7861,0.7342
33,0.5524,0.8032,0.6697,0.7717
34,0.5456,0.8073,0.6471,0.7774
35,0.5400,0.8112,0.6502,0.7802
36,0.5342,0.8121,0.6719,0.7690
37,0.5303,0.8137,0.7376,0.7501
38,0.5238,0.8151,0.6636,0.7728
39,0.5192,0.8155,0.6529,0.7789
40,0.5208,0.8168,0.6711,0.7710
41,0.5170,0.8180,0.6704,0.7742
42,0.5125,0.8197,0.6607,0.7755
43,0.5138,0.8217,0.6502,0.7806
44,0.5079,0.8208,0.6588,0.7796
45,0.5028,0.8209,0.6770,0.7731
46,0.4992,0.8242,0.6615,0.7756
47,0.4980,0.8241,0.6829,0.7688
48,0.4969,0.8229,0.6494,0.7821
49,0.4930,0.8269,0.6508,0.7795
50,0.4907,0.8262,0.6490,0.7821

```

3.2 不同卷积核大小

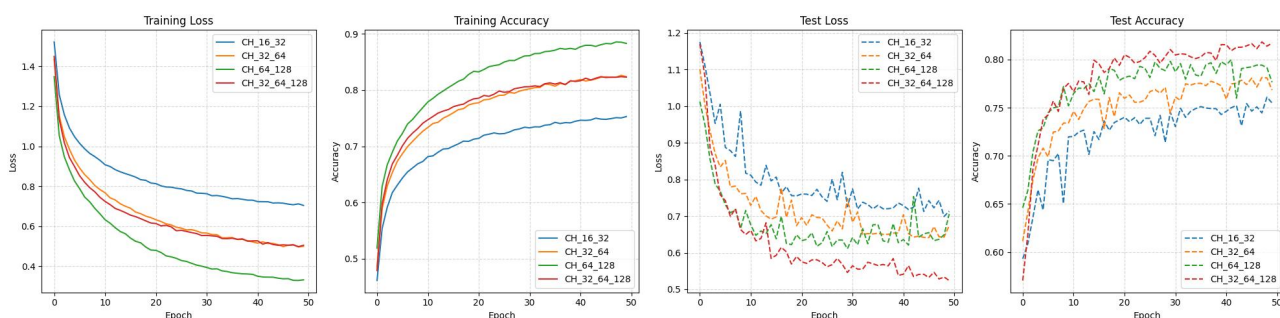
使用包含不同卷积核大小的网络进行训练，各网络的其余网络结构与训练条件均保持一致，仅调整卷积层的 `kernel_size` 参数以比较其对特征提取能力与分类性能的影响。设置了三组配置：卷积核大小分别为 3×3 (KS_3x3)、 5×5 (KS_5x5) 和 7×7 (KS_7x7)。三种网络在相同数据集、相同训练轮数、batch size、学习率以及其他超参数不变的条件下进行训练，并对训练过程中损失与准确率的变化趋势，以及最终在测试集上的分类效果进行对比分析，结果如下图所示：



从实验结果可以看出，不同卷积核大小对模型的训练动态与泛化性能产生了一定影响。三种卷积核配置（ 3×3 、 5×5 、 7×7 ）在训练过程中均能够稳定收敛，训练损失持续下降，训练准确率逐步提升，表明模型均具备有效的特征学习能力。其中， 5×5 和 7×7 卷积核在中后期的训练准确率略高于 3×3 ，但差异并不显著。在测试集上，不同卷积核大小的模型最终测试准确率均稳定在约 77%–79% 区间，整体性能接近，但较大的卷积核在测试阶段表现出更明显的波动，尤其是 7×7 卷积核，其测试损失和准确率曲线起伏较大，说明模型对训练过程更为敏感，泛化稳定性相对较弱。相比之下， 3×3 和 5×5 卷积核在测试集上的表现更加平稳，其中 5×5 卷积核在准确率和稳定性之间取得了较为均衡的效果。综合来看，在 CIFAR-10 这类低分辨率图像分类任务中，采用较小或中等尺寸的卷积核更有利于获得稳定且具有良好泛化能力的模型表现。

3.3 不同通道数

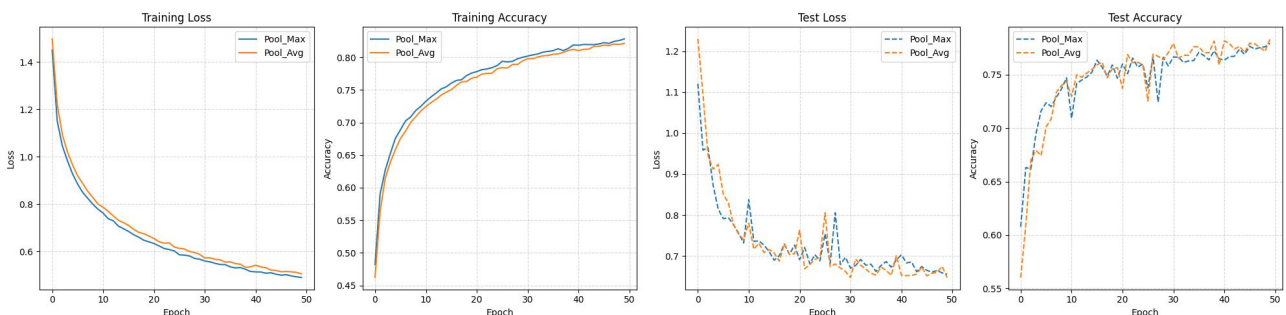
保持使用包含不同卷积通道数配置的网络进行训练，各网络的其余结构与训练条件均保持一致，仅调整卷积层的 `conv_channels` 参数，以比较网络容量变化对特征表达能力与分类性能的影响。本实验设置了四组配置：两层卷积且通道数较小的 [16, 32]、默认规模的 [32, 64]、更大规模的 [64, 128]，以及在默认基础上增加一层卷积、形成三层卷积结构的 [32, 64, 128]。四种网络在相同条件下完成训练，训练过程中的性能变化和分类效果如下图所示：



从实验结果可以看出，卷积层通道数的增加显著提升了模型的特征表达能力与整体性能。随着通道数由 [16, 32] 增加至 [32, 64] 和 [64, 128]，模型在训练集上的损失下降速度明显加快，最终训练准确率持续提升，表明更大的通道数能够学习到更加丰富和判别性更强的特征表示。在测试集上，较小通道数配置 [16, 32] 的测试损失较高、准确率较低，整体泛化能力受限；中等规模的 [32, 64] 配置在性能和稳定性之间取得了较为均衡的结果；而通道数进一步增大的 [64, 128] 以及三层卷积结构的 [32, 64, 128] 在测试准确率上整体优于前两种配置，其中 [32, 64, 128] 在测试损失最低、测试准确率最高，表现出最佳的泛化性能。综合来看，适当增加卷积通道数及网络深度有助于提升 CIFAR-10 图像分类性能，但同时也增加了模型复杂度，对计算资源与训练稳定性提出了更高要求。

3.4 不同池化方式

使用包含不同池化方式的网络进行训练，各网络的其余结构与训练条件均保持一致，仅调整卷积模块中的 `pool_type` 参数，以比较不同下采样策略对特征保留能力与分类性能的影响。本实验设置了两组配置：采用最大池化的 `Pool_Max` 与采用平均池化的 `Pool_Avg`。两种网络在相同条件下完成训练，并对训练/测试损失与准确率的变化趋势及最终分类效果进行对比分析，训练过程中的性能变化和分类效果如下图所示：

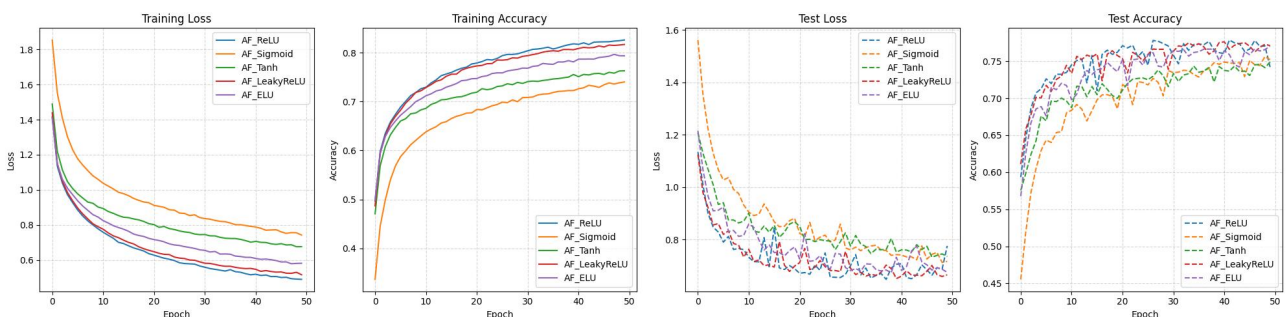


从实验结果可以看出，不同池化方式对模型整体性能的影响相对有限，但仍

表现出一定差异。采用平均池化（Average Pooling）的网络在训练与测试阶段的损失和准确率变化趋势与最大池化整体相近，但在训练初期和中期的准确率略低，说明平均池化在保留整体特征分布的同时，可能削弱了局部强响应特征的判别性。在测试集上，两种池化方式的最终准确率接近，均稳定在约 76%–78% 区间，泛化性能差异不明显。

3.5 不同激活函数

使用包含不同激活函数的网络进行训练，各网络的其余结构与训练条件均保持一致，仅调整网络中的 `activation` 参数，以比较不同非线性变换方式对模型收敛速度、训练稳定性与最终分类性能的影响。本实验设置了五组配置：分别采用 ReLU（默认）、Sigmoid、Tanh、LeakyReLU 以及 ELU。五种网络在相同条件下完成训练，并对最终分类效果进行对比分析：

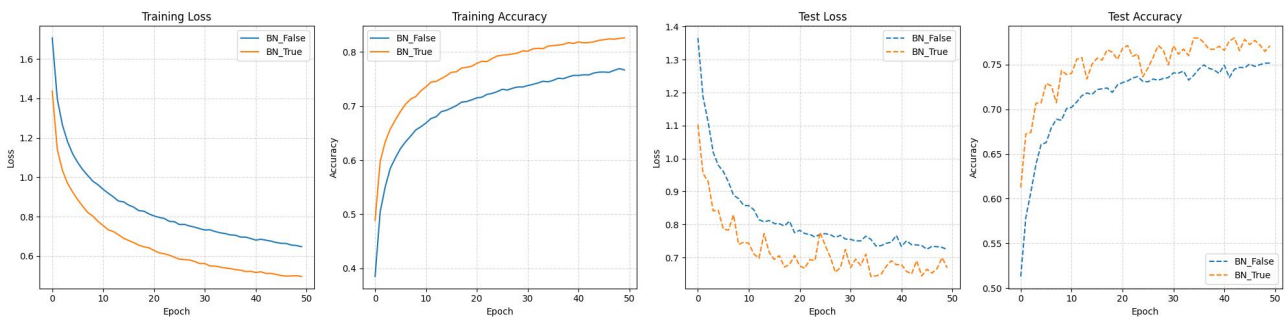


从实验结果可以看出，不同激活函数对模型的收敛速度、训练稳定性以及最终分类性能具有显著影响。整体而言，采用 ReLU、LeakyReLU 和 ELU 的网络在训练过程中损失下降更快、训练准确率提升更为迅速，表现出良好的优化特性与数值稳定性。相比之下，采用 Sigmoid 激活函数的网络在训练初期收敛明显较慢，训练损失和测试损失均较高，训练与测试准确率始终低于其他配置，反映出 Sigmoid 在深层网络中易出现梯度饱和、优化效率较低的问题。Tanh 激活函数的整体表现优于 Sigmoid，但在收敛速度和最终准确率方面仍略逊于 ReLU

类激活函数。ELU 在训练稳定性和性能上表现居中，能够取得较为平衡的结果。综合来看，对于本实验中的 CIFAR-10 图像分类任务，ReLU 及其改进形式（LeakyReLU、ELU）更适合作为卷积神经网络的激活函数选择。

3.6 是否引入批归一化（BatchNorm）

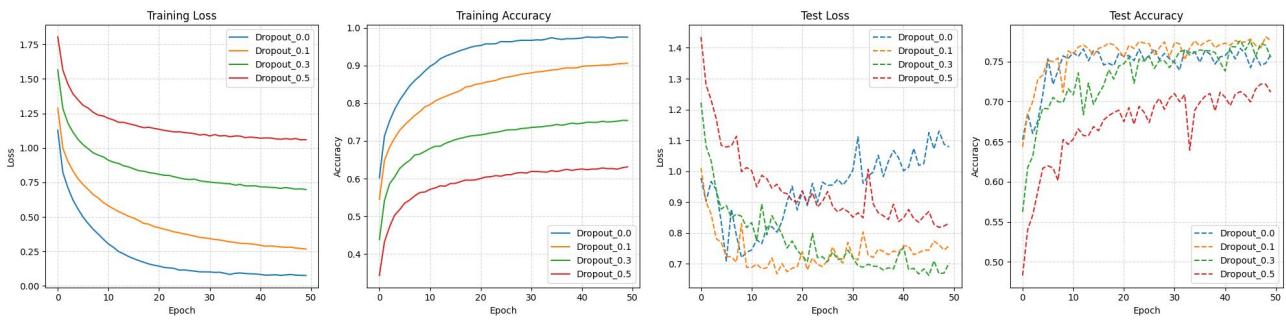
使用是否引入批归一化（Batch Normalization）的网络进行训练，各网络的其余结构与训练条件均保持一致，仅调整模型中的 `use_bn` 参数，以分析 BatchNorm 对训练收敛速度、数值稳定性以及最终分类性能的影响。本实验设置了两组配置：不使用 BatchNorm 与使用 BatchNorm。两种网络在相同条件下完成训练，并对最终分类效果进行对比分析：



从实验结果可以明显看出，引入批归一化（Batch Normalization）对模型训练过程和最终性能具有显著的正向影响。使用 BatchNorm 的模型在训练初期损失下降更快，训练准确率提升更为迅速，表现出更好的收敛速度与优化稳定性。在测试集上，引入 BatchNorm 的网络整体测试损失更低，测试准确率始终高于未使用 BatchNorm 的配置，且波动幅度更小，说明批归一化在缓解内部协变量偏移、提升模型泛化能力方面发挥了重要作用。同时，BatchNorm 对训练过程具有一定的正则化效果，使模型在较高训练精度下仍能保持较好的测试性能。综合来看，在本实验的 CIFAR-10 图像分类任务中，引入批归一化能够显著提升模型的训练效率、稳定性以及最终分类性能，是一种行之有效的结构改进手段。

3.7 不同的 Dropout 比率

使用包含不同 Dropout 比率的网络进行训练，各网络的其余结构与训练条件均保持一致，仅调整模型中的 `dropout_rate` 参数，以分析 Dropout 强度对模型过拟合程度、训练收敛速度以及最终泛化性能的影响。本实验设置了四组配置：不使用 Dropout、低强度 Dropout（`dropout_rate=0.1`）、中等强度 Dropout（`dropout_rate=0.3`），以及高强度 Dropout（`dropout_rate=0.5`）。四种网络在相同条件下完成训练，并对训最终分类效果进行对比分析：

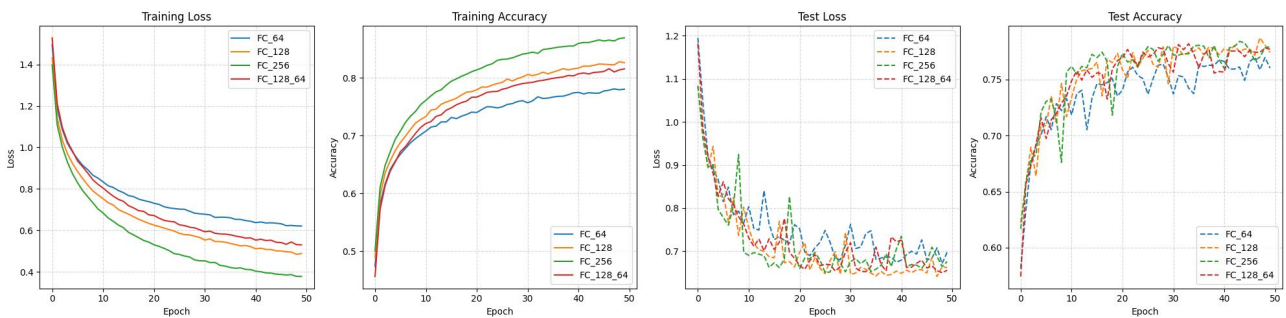


从实验结果可以看出，Dropout 比率对模型的训练拟合能力与泛化性能产生了显著影响。未使用 Dropout（`dropout_rate=0.0`）的网络在训练过程中损失下降最快、训练准确率最高，最终几乎可以完全拟合训练数据，但其测试损失较高且波动明显，存在一定程度的过拟合现象。当引入较小比例的 Dropout（`dropout_rate=0.1`）后，训练损失和训练准确率略有下降，但测试损失明显降低、测试准确率提升并趋于稳定，表现出更好的泛化能力，整体效果最为均衡。中等强度 Dropout（`dropout_rate=0.3`）进一步抑制了过拟合，但同时也限制了模型的表达能力，导致训练准确率和测试准确率均有所下降。在高强度 Dropout（`dropout_rate=0.5`）条件下，模型训练过程明显受限，收敛速度较慢，训练准确率较低，测试性能亦不理想，表现出欠拟合的趋势。综合来看，适度的 Dropout 有助于提升模型泛化性能，其中较低比例（如 0.1）在本实验设置下能够在抑制

过拟合与保持模型表达能力之间取得较优平衡。

3.8 不同的全连接层大小

使用包含不同全连接层规模配置的网络进行训练，各网络的卷积部分结构与训练条件均保持一致，仅调整分类头（全连接层）的 `fc_sizes` 参数，以比较全连接层容量与层数变化对特征融合能力、模型拟合程度及最终分类性能的影响。本实验设置了四组配置：单隐藏层且维度为 64 的 `FC_64`、默认规模的 `FC_128`、更大规模的 `FC_256`，以及采用两层全连接结构的 `FC_128_64`。四种网络在相同条件下完成训练，并对最终分类效果进行对比分析：



从实验结果可以看出，全连接层的规模与层数对模型的拟合能力和最终分类性能具有一定影响。随着全连接层维度由 64 增大到 128 和 256，模型在训练集上的损失下降速度加快，训练准确率持续提升，表明更大的全连接层能够更充分地利用卷积层提取的高层特征，提高模型的表达能力。在测试集上，不同全连接层配置的最终准确率整体差异不大，均稳定在约 75%–78% 区间，但中等规模的全连接层在测试损失和准确率上略优于 `FC_64`，表现出更好的泛化性能。采用两层全连接结构的 `FC_128_64` 在训练和测试表现上介于单层 128 和 256 之间，未带来明显的性能提升，说明在当前网络深度与数据规模下，简单增加全连接层层数的收益有限。综合来看，适当增大全连接层维度有助于提升模型性能，但过度复杂的分类头并不会显著改善泛化效果。

4. 总结与体会

本次基于 CIFAR-10 数据集的卷积神经网络实验，系统地完成了 CNN 模型的设计、训练以及多种结构与超参数的对比分析，我对卷积神经网络在图像分类任务中的工作机制和性能影响因素有了更加直观和深入的理解。

从各组对比实验可以看出，不同网络结构和超参数对模型性能的影响具有明显差异。其中，卷积核大小和池化方式对最终性能的影响相对有限，但会影响模型的训练稳定性；适中尺寸的卷积核（如 3×3 或 5×5 ）更适合 CIFAR-10 这类低分辨率图像任务。卷积通道数和网络深度对模型性能影响较为显著，适当增加通道数和层数能够有效提升特征表达能力和分类准确率，但同时也增加了模型复杂度和计算开销，需要在性能与效率之间进行权衡。

在训练策略与正则化手段方面，实验清晰地验证了 Batch Normalization 和 Dropout 的重要作用。BatchNorm 能显著加快模型收敛速度、提升训练稳定性并改善泛化性能；而 Dropout 在防止过拟合方面效果明显，但其强度需要合理设置，过大的 Dropout 比率会导致模型欠拟合。激活函数实验结果表明，ReLU 及其改进形式（如 LeakyReLU、ELU）在收敛速度和最终性能上均优于 Sigmoid 和 Tanh，更适合用于深层卷积神经网络。

本次实验不仅加深了我对 CNN 各组成模块及其作用机理的理解，也强化了我对“模型性能来源于结构设计 with 训练策略协同作用”的认识。为后续更复杂网络（如更深 CNN 或 ResNet 等）的学习和应用奠定了良好的实践基础。